

Project Title:

Micro-controlled Multi-Mode cooling system

Course Name & Number:

Embedded System CCE407

Date Due:

5/14/2026

Date Handed In:

5/14/2026

Team Members:

#	Student Name	Student ID	Section
1	Ramy Said Eliwa Elsayed	231903795	لائحة قديمة
2	David George Helmy Ekladious	231903654	لائحة قديمة
3	Ahmed Mohamed Abdelrazik	232903959	لائحة قديمة
4	Youssef Amer Sayed Mostafa	231903616	لائحة قديمة
5	Salma Gamal Saadoun Ahmed	221903217	لائحة قديمة

Table of Contents

Table of Contents.....	1
1. Introduction.....	2
2. Project Significance.....	2
3. System Features.....	3
4. Hardware Components.....	4
5. Schematic Diagram.....	5
6. Hardware Assembly & Wiring.....	6
6.1. Microcontroller Core Setup (AT89S52).....	6
6.2. LCD Interfacing (16x2 Display).....	6
6.3. Matrix Keypad Integration (4x4).....	7
6.4. Motor Driver Isolation (L293D).....	7
6.5. Serial Communication & Power Routing (CH340G Adapter).....	7
7. System Methodology.....	8
7.1. Initialization Phase (Pre-Loop).....	8
7.2. Input Scanning (The Super Loop Begins).....	8
7.3. Action Execution & Status Mirroring.....	9
8. Challenges and Troubleshooting.....	10
8.1. Power Budgeting and Thermal Inefficiency.....	10
8.2. LCD Initialization and Contrast Calibration Failure.....	10
8.3. Matrix Keypad Signal Routing Conflicts.....	10
9. Source Code.....	11
10. Conclusion.....	18
11. References.....	18

1. Introduction

This report details the design, implementation, and evaluation of a Smart Fan Control System, an advanced embedded application engineered around the AT89S52 microcontroller (based on the classic 8051 architecture). The fundamental objective of this project was to transform a standard, single-speed DC cooling fan into an intelligent, interactive thermal management device. To achieve this, the system bridges software logic with physical hardware to offer both manual control and automated operational modes.

At its core, the project demonstrates how multiple peripheral devices can be orchestrated simultaneously by a single processing unit. We integrated a 4x4 matrix keypad to capture precise user commands, translating physical button presses into digital instructions. An alphanumeric 16x2 Liquid Crystal Display (LCD) was implemented to provide localized, instant visual feedback regarding the fan's current speed and active timer modes. Furthermore, a UART-based serial bridge (utilizing a CH340G adapter) was established to transmit live operational data to a connected laptop, allowing for real-time monitoring and diagnostics. Ultimately, this system provides users with highly granular control over fan velocity and operational duration, serving as a comprehensive prototype that mirrors the complexities of modern industrial and smart-home environmental control systems.

2. Project Significance

This project serves as a comprehensive practical application of embedded system design. It highlights the integration of various peripherals (Input/Output devices, Motor Drivers, Displays) and communication protocols. The importance of this project lies in bridging the gap between hardware execution and software logic, specifically demonstrating Software PWM (Pulse Width Modulation) for speed control and UART (Universal Asynchronous Receiver-Transmitter) for PC-microcontroller interfacing.

3. System Features

The system was architected with advanced, user-centric functionalities to showcase the capabilities of the 8051 microcontroller. Here is a detailed breakdown of its core features:

- **Multi-Speed Control via Software PWM:** Instead of a simple binary on/off switch, the system implements **Software Pulse Width Modulation (PWM)** to achieve 6 distinct fan speed levels. **Since the standard 8051 lacks dedicated hardware PWM pins, this is achieved programmatically.** The software rapidly toggles the motor driver's enable pin at calculated intervals, giving the user highly granular control over the airflow intensity.
- **Interactive Real-Time LCD Interface:** A robust 16x2 Alphanumeric LCD serves as the primary local user interface (UI). It actively updates to display the current operational mode (Continuous vs. Timer), the active fan speed (1 to 6), and live countdowns. This provides immediate, localized visual feedback, eliminating any ambiguity about the system's internal state.
- **Real-Time Serial Monitoring (PC Terminal): [KEY FEATURE]** The system utilizes a **CH340G USB-to-TTL adapter** to establish a direct, full-duplex UART connection with a laptop. Acting as a real-time diagnostic tool, every localized event—such as keypad inputs, fan speed adjustments, and timer expirations—is instantly mirrored as formatted text strings (e.g., "Speed changed to: 3") directly to the Arduino IDE Serial Monitor. This effectively transforms the laptop into a live data-logging and tracking station for the embedded microcontroller.
- **Dual-Purpose USB Interface (Power & Data):** In a highly efficient hardware design choice, the CH340G module acts as a Power Delivery Network (PDN) in addition to handling data transmission. By tapping into the steady 5V/500mA supply of the laptop's USB port, the project entirely bypasses the need for bulky external 9V batteries and linear voltage regulators. This ensures thermal stability and maintains a clean, compact breadboard footprint.
- **Automated Timer Modes:** The firmware includes logic for pre-programmed countdown timers (5, 10, and 15-second intervals). Once engaged via dedicated keypad keys ('A', 'B', 'C'), the microcontroller enters an automated countdown state, displaying the remaining time on the LCD, and safely cutting power to the motor driver upon expiration without requiring further manual intervention.

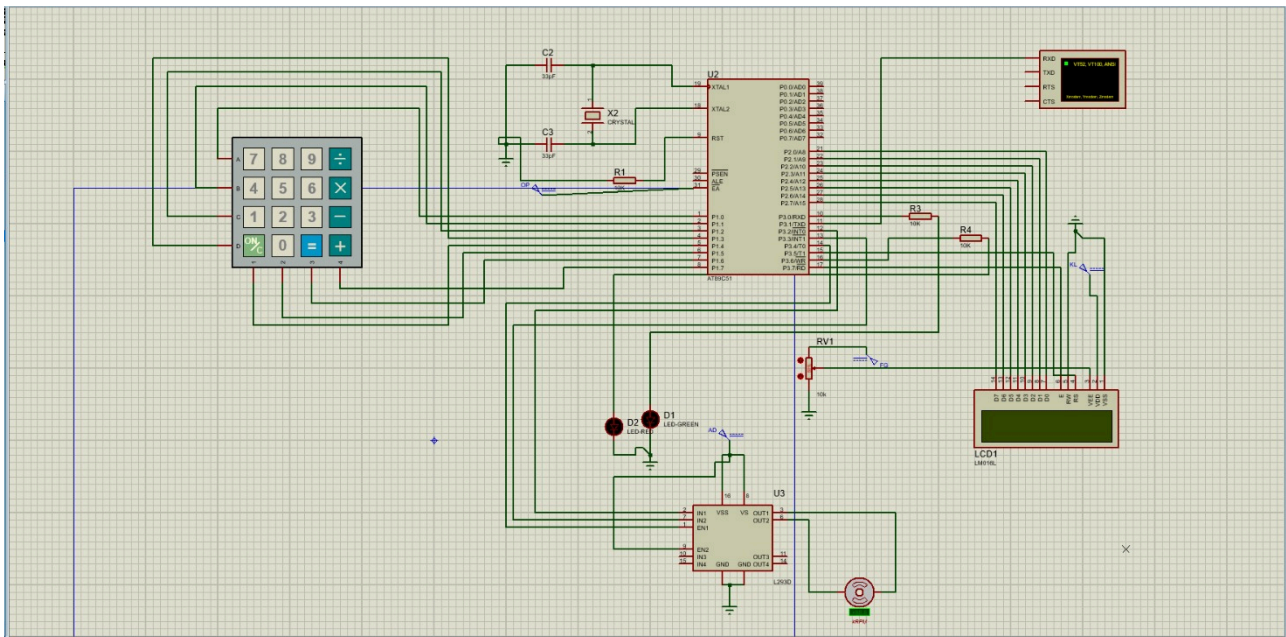
4. Hardware Components

The project utilizes the following electronic components:

Component	Description
Microcontroller	AT89S52-24PU (8051 Architecture) with 40-Pin DIP Base.
Clock	11.0592 MHz Crystal Oscillator with 2x 33pF Ceramic Capacitors.
Display	16x2 Blue Backlight LCD.
Contrast Control	10K Potentiometer (RV1) for LCD contrast adjustment.
Input Device	4x4 Matrix Membrane Keypad.
Motor	5V DC Fan.
Driver	L293D Motor Driver IC.
Serial Communication & Power	CH340G USB to TTL Adapter (Used for both PC interfacing and providing a steady 5V power supply).
Indicators (LEDs)	Red LED: Indicates system is powered on but fan is in Stop/Standby mode. Green LED: Illuminates when the fan is actively running, providing a quick visual cue of the motor's operational state without needing to read the LCD.
Resistors	10K Ohm Resistor: Used in the 8051's reset circuit and as pull-up resistors. 330 Ohm (or 220 Ohm) Resistors: Act as current-limiting resistors connected in series with the LEDs to prevent burnout or damage to the MCU.
Jumper Wires	Assorted DuPont cables (Male-to-Male and Male-to-Female) to establish physical electrical pathways across the breadboard. They reliably route data signals and distribute power (5V and GND).
Capacitors	10uF Electrolytic Capacitor: For the reset circuit pulse. 33pF Ceramic Capacitors: To stabilize the crystal oscillator.
Power Supply	The traditional battery was excluded. Logic power (5V) is sourced entirely from the laptop via the CH340G module.

5. Schematic Diagram

Below is the hardware schematic designed in Proteus, showing the wiring between the MCU, LCD, Keypad, Motor Driver, and the Virtual Terminal representing the CH340G connection.



6. Hardware Assembly & Wiring

The entire prototype was meticulously assembled on a standard **Breadboard**. Careful attention was paid to power distribution, signal isolation, and correct pin mapping to ensure stable operation. Below is the detailed breakdown of the hardware interfacing:

6.1. Microcontroller Core Setup (AT89S52)

The 8051 MCU was placed centrally across the breadboard's dividing trench.

- **Power:** Pin 40 (**VCC**) was tied to the positive 5V rail, and Pin 20 (**GND**) was tied to the negative rail. Pin 31 (**EA/VPP** - External Access) was explicitly hardwired to 5V (**VCC**) to force the microcontroller to execute the program from its internal Flash memory rather than looking for an external ROM.
- **Reset Circuit:** Connected to Pin 9 (**RST**). A 10uF electrolytic capacitor was connected between VCC and Pin 9, and a 10K resistor from Pin 9 to Ground. This provides a clean "Power-On Reset" pulse when the system boots up.
- **Clock/Oscillator:** An 11.0592 MHz crystal was connected across pins 18 (**XTAL2**) and 19 (**XTAL1**). To filter out high-frequency noise and stabilize the clock signal, two 33pF ceramic capacitors were connected from each crystal leg to Ground. *Note: 11.0592 MHz was specifically chosen over 12 MHz because it divides perfectly to generate standard UART baud rates like 9600 bps without error.*

6.2. LCD Interfacing (16x2 Display)

The LCD was configured to operate in **8-Bit Data Mode** to maximize data transfer speed.

- **Data Bus:** The 8 data pins of the LCD (**D0** to **D7**) were connected sequentially to the entire Port 2 of the MCU (**P2.0** to **P2.7**).
- **Control Pins:** The Register Select (**RS**) was wired to **P3.5**, and the Enable (**EN**) pin to **P3.7**. The Read/Write (**R/W**) pin was hardwired directly to Ground because the MCU only ever *writes* data to the LCD in this project; it never needs to *read* from it.
- **Power & Contrast:** Pins 1 and 2 of the LCD were connected to Ground and 5V respectively. The 10K Potentiometer was wired with its outer legs to 5V and Ground, and its middle (wiper) leg connected to LCD Pin 3 (**V0**) to provide precise voltage adjustment for the screen's contrast. Backlight pins (15 & 16) were powered with a series current-limiting resistor to prevent burnout.

6.3. Matrix Keypad Integration (4x4)

The keypad utilizes a row-column scanning matrix, requiring 8 digital I/O pins.

- **Mapping:** The 8 outputs of the keypad were routed sequentially to **Port 1** of the MCU. Rows 1-4 were connected to **P1.0 - P1.3**, and Columns 1-4 to **P1.4 - P1.7**.
- We took advantage of Port 1's internal pull-up resistors, which naturally hold the pins at a logic HIGH (5V) state until a button press connects a row to a column, driving the signal LOW (0V) for the software to detect.

6.4. Motor Driver Isolation (L293D)

The L293D IC is crucial. A microcontroller cannot supply enough current (Amps) to drive a mechanical DC fan directly; doing so would fry the MCU. The L293D acts as a high-current switch.

- **Power:** Pin 16 (**VSS** - Logic Power) and Pin 8 (**VS** - Motor Power) were both connected to the 5V rail. The four middle pins (4, 5, 12, 13) were all wired to Ground. Tying all grounds together is vital for the L293D as these pins also act as internal heat sinks.
- **Logic to Motor:** MCU pins **P3.2**, **P3.3**, and **P3.4** were connected to the driver's Input 1, Input 2, and Enable 1 pins. The fan's terminals were connected to Output 1 and Output 2. By sending a PWM signal to the Enable pin, the L293D rapidly switches the high-current 5V power to the fan, regulating its speed.

6.5. Serial Communication & Power Routing (CH340G Adapter)

This module acts as the physical bridge between the embedded hardware and the laptop, serving a dual and highly critical purpose:

1. **Cross-Over Data Wiring (UART):** To establish communication, the transmitter must talk to the receiver. Therefore, the CH340G's **RX** pin was connected to the MCU's **TX** pin (**P3.1**), and the CH340G's **TX** pin was connected to the MCU's **RX** pin (**P3.0**).
2. **System Power Injection:** The **VCC** (5V) pin of the CH340G was connected directly to the positive power rail of the breadboard, and the **GND** pin to the negative rail.
 - *Why this matters:* The common ground unifies the voltage reference for both the UART data signals and the logic ICs. Furthermore, it completely powered the AT89S52, the LCD, the LED indicators, and the small 5V DC fan directly from the laptop's USB port, cleanly eliminating the need for bulky external battery packs or voltage regulators.

7. System Methodology

The software architecture of this project is based on a **Super Loop Architecture**. This means the microcontroller executes a continuous, never-ending `while(1)` loop. Within this loop, it constantly reads inputs, updates states, and drives hardware. Here is our detailed breakdown of the execution flow:

7.1. Initialization Phase (Pre-Loop)

Before the system can interact with the user, it must configure its internal hardware modules and external peripherals.

- **UART (Universal Asynchronous Receiver Transmitter) Setup:** The `uart_init()` function configures Timer 1 in Mode 2 (**8-bit auto-reload**). Using an 11.0592 MHz crystal, assigning `0xFD` to `TH1` mathematically generates a precise baud rate of **9600 bps**.
- **LCD Setup:** Standard initialization commands are sent to prepare the LCD.
- **Boot Message:** Finally, a string is sent through the CH340G to the laptop to confirm a successful boot.

```
void main() {  
    char key;  
    uart_init(); // Sets TH1=0xFD, SCON=0x50, starts Timer 1  
    lcd_init(); // Clears screen, sets 8-bit mode  
  
    // Initial hardware states  
    IN1 = 1; IN2 = 0; EN1 = 0; // Motor off  
    GREEN = 0; RED = 1;      // Standby LED on  
  
    // Send test message to PC Terminal  
    uart_msg("System Ready...\r\n");  
  
    // Enter the Super Loop...
```

7.2. Input Scanning (The Super Loop Begins)

Once inside the `while(1)` loop, the very first task in every cycle is to check if the user

is pressing a button. The `keypad_scan()` function sweeps the rows and columns. It includes **debouncing logic**—a critical software technique that ignores electrical noise and ensures that one physical press translates to exactly one software command, preventing false or multiple triggers.

```
while(1) { // The Super Loop
    key = keypad_scan(); // Continuously scan for user input
```

7.3. Action Execution & Status Mirroring

When a key is detected, the code uses conditional statements (`if / else if`) to decide the action. A key feature of this system is **Mirroring**: Any state change must be reflected locally (on the LCD) AND remotely (on the PC terminal).

For example, if the user presses a speed button ('1' to '6'):

1. **Variable Update:** The internal state variable `speed_val` is updated.
2. **Local Display:** The LCD is commanded to show the new speed.
3. **Serial Transmission:** The `uart_msg()` function streams the exact same context through the CH340G to the laptop.

```
// If a Speed Button (1-6) is pressed
```

```
if (key >= '1' && key <= '6') {
```

```
    speed_val = key; // 1. Update internal state
```

```
    is_timer_mode = 0;
```

```
// 2. Mirror to Local Display (LCD)
```

```
lcd_cmd(0x80); lcd_string("Speed Set: "); lcd_data(speed_val);
```

```
lcd_cmd(0xC0); lcd_string("Mode: Continuous");
```

```
// 3. Mirror to Remote Terminal (PC via CH340G)
```

```
uart_msg("Speed changed to: ");
```

```
uart_tx(speed_val);
```

```
uart_msg("\r\n");
```

```
}
```

8. Challenges and Troubleshooting

During the physical prototyping and breadboard assembly phase, several critical engineering challenges were encountered. By applying systematic debugging and root-cause analysis, we successfully resolved them:

8.1. Power Budgeting and Thermal Inefficiency

- **The Problem:** The core components (8051 MCU, LCD Backlight, logic ICs) require a strict 5V DC reference. Standard 9V alkaline batteries were initially considered. However, stepping down 9V to 5V using a linear voltage regulator (like the LM7805) causes severe thermal inefficiency. The excess voltage (4V) is dissipated as heat, which would require a heatsink, consume valuable breadboard space, and drain the battery capacity rapidly under the load of the motor.
- **The Engineered Solution:** We entirely eliminated the battery and regulator circuit by utilizing the **CH340G USB-to-TTL module as a Power Delivery Network (PDN)**. Since USB 2.0 ports reliably provide up to 500mA at 5V, we routed the **VCC** and **GND** pins from the CH340G directly to the breadboard's power rails. This innovative approach solved the thermal issues, bypassed the need for bulky regulators, and provided an exceptionally stable logic voltage reference, all while simultaneously maintaining our UART data connection.

8.2. LCD Initialization and Contrast Calibration Failure

- **The Problem:** Upon initial power-up, the 16x2 LCD failed to initialize properly, displaying only a single row of solid black blocks—a classic hardware error state indicating that the display logic is powered but not receiving valid data or correct voltage.
- **The Troubleshooting Process:** The debugging was divided into two stages. First, physical continuity checks using a multimeter revealed high resistance and loose connections in the DuPont jumper wires across the data bus (Port 2). Replacing these secured the transmission lines. Second, the analog contrast voltage at pin **V0** was out of bounds. We precisely calibrated the 10K trimmer potentiometer, effectively modifying the voltage divider output until it reached the correct operating threshold for the liquid crystals to become visible.

8.3. Matrix Keypad Signal Routing Conflicts

- **The Problem:** After rectifying the LCD, the system registered erroneous inputs from the 4x4 keypad (e.g., pressing '1' resulted in '4' on the terminal).
- **The Root Cause:** This was traced back to a physical routing mismatch rather than a software bug. During the rewiring of the LCD, the sequential mapping of the keypad's rows and columns to the MCU's Port 1 (P1.0 through P1.7) was physically inverted on the breadboard.
- **The Solution:** Instead of rewriting the hardcoded 2D array and scanning logic in the embedded C software, the physical hardware layout was meticulously re-traced and corrected pin-by-pin. This ensured that the physical row/column matrix flawlessly mirrored the software's scanning algorithm, restoring precise user input functionality.

9. Source Code

The following is the core C-language implementation used to program the AT89S52 microcontroller:

```
#include <reg52.h>

#define LCD P2

sbit RS = P3^5;
sbit EN_LCD = P3^7;
sbit GREEN = P3^0;
sbit RED = P3^6;
sbit IN1 = P3^2;
sbit IN2 = P3^3;
sbit EN1 = P3^4;

unsigned char speed_val = '1';
unsigned int runtime = 0;
bit fan_running = 0;
bit is_timer_mode = 0;

// Function Prototypes
void delay(unsigned int ms);
void lcd_cmd(unsigned char cmd);
void lcd_data(unsigned char dat);
void lcd_string(char *str);
void lcd_init();
void uart_init();
void uart_tx(char ch);
```

```

void uart_msg(char *str);

char keypad_scan();

void motor_control();

void main() {
    char key;

    uart_init(); // Initialize Serial Comm
    lcd_init();  // Initialize LCD

    IN1 = 1; IN2 = 0; EN1 = 0;

    GREEN = 0; RED = 1;

    uart_msg("System Ready...\r\n");

    while(1) {
        key = keypad_scan();

        // Speed Buttons
        if (key >= '1' && key <= '6') {
            speed_val = key;

            is_timer_mode = 0;

            lcd_cmd(0x80); lcd_string("Speed Set: "); lcd_data(speed_val);
            lcd_cmd(0xC0); lcd_string("Mode: Continuous");

            uart_msg("Speed changed to: "); uart_tx(speed_val);
            uart_msg("\r\n");
        }

        // Timer Buttons
        else if (key == 'A' || key == 'B' || key == 'C') {
            is_timer_mode = 1;

```

```

        if (key == 'A') { runtime = 500; lcd_cmd(0xC0);
lcd_string("Timer: 5 sec"); }

        if (key == 'B') { runtime = 1000; lcd_cmd(0xC0);
lcd_string("Timer: 10 sec"); }

        if (key == 'C') { runtime = 1500; lcd_cmd(0xC0);
lcd_string("Timer: 15 sec"); }

    }

    // ON Button
    else if (key == '*') {

        fan_running = 1;

        GREEN = 1; RED = 0;

        lcd_cmd(0x01); lcd_string("Fan ON...");

        uart_msg("Fan Started\r\n");

    }

    // OFF Button
    else if (key == 'D') {

        fan_running = 0;

        EN1 = 0; GREEN = 0; RED = 1;

        lcd_cmd(0x01); lcd_string("Fan Stopped");

        uart_msg("Fan Stopped Manually\r\n");

        delay(500);

    }


    // Motor Logic
    if(fan_running) {

        motor_control();

        if(is_timer_mode) {

            if (runtime > 0) runtime--;

```



```

void motor_control() {
    unsigned char duty;
    if (speed_val == '1') duty = 2;
    else if (speed_val == '2') duty = 4;
    else if (speed_val == '3') duty = 6;
    else if (speed_val == '4') duty = 7;
    else if (speed_val == '5') duty = 8;
    else duty = 9;

    EN1 = 1; delay(duty);
    EN1 = 0; delay(10 - duty);
}

void lcd_cmd(unsigned char cmd) {
    LCD = cmd; RS = 0; EN_LCD = 1; delay(1); EN_LCD = 0;
    if(cmd == 0x01) delay(20);
}

void lcd_data(unsigned char dat) {
    LCD = dat; RS = 1; EN_LCD = 1; delay(1); EN_LCD = 0;
}

void lcd_init() {
    lcd_cmd(0x38); lcd_cmd(0x0C); lcd_cmd(0x01); lcd_cmd(0x06);
}

```

```
void lcd_string(char *str) {  
    while(*str) lcd_data(*str++);  
}  
  
void delay(unsigned int ms) {  
    unsigned int i, j;  
    for(i=0; i<ms; i++)  
        for(j=0; j<112; j++);  
}
```

10. Conclusion

In conclusion, the Smart Fan Control System successfully demonstrates the seamless integration of embedded software logic with practical hardware design to create a fully functional, interactive cooling device. The project achieved its core objectives by transforming theoretical microcontroller concepts into a tangible prototype.

From a software perspective, the implementation of a Super Loop architecture combined with Software Pulse Width Modulation (PWM) proved highly effective. It allowed for precise, multi-level motor speed control despite the standard 8051 architecture's lack of dedicated PWM hardware. Furthermore, the intelligent integration of digital I/O multiplexing enabled the reliable scanning of the 4x4 matrix keypad alongside real-time state updates on the LCD.

From a hardware perspective, the project overcame significant physical assembly and power-routing challenges. The most critical innovation was the dual-purpose utilization of the CH340G USB-to-TTL module. By leveraging this module not only for establishing a full-duplex UART communication bridge for live PC monitoring but also as the primary 5V Power Delivery Network, the design achieved high thermal efficiency. This elegant solution completely eliminated the need for external batteries and bulky voltage regulators, ensuring an optimized and robust physical architecture.

Ultimately, the system met all initial design specifications, delivering reliable manual speed control, automated timer shutdowns, and real-time remote diagnostics. This stable baseline architecture establishes a highly scalable foundation, paving the way for future enhancements such as integrating closed-loop temperature sensors for automated dynamic speed adjustments or appending Wi-Fi modules for comprehensive IoT smart-home integration.

11. References

1. AT89S52 Microcontroller Datasheet (Atmel).
2. L293D Motor Driver Datasheet (Texas Instruments).
3. HD44780 LCD Controller Manual.
4. CH340G USB to Serial Driver Documentation.